SMARTAG OPS ADVISOR

Master Document

Scope, roles, architecture, features, data, API, configuration, security and operations

25 October 2026 · Repository hajmalirfan/Data-Drive

This document describes the application as implemented in the repository. It is intended for developers, reviewers, operators, and deployers. Configuration examples use placeholders; credentials belong in local environment files or a secret manager, never in this document.

Last updated 25 Oct 2026 (commit `3b0677d`). The history of how each feature was requested and built is in the companion Prompt Document.

Contents

1. Executive summary
2. Product scope and roles
3. Architecture
4. Repository map
5. User-facing features
6. Data model
7. API reference
8. Configuration reference
9. Local development
10. Data and external services
11. Security and operations
12. Build and validation
13. Known constraints and maintenance notes

1. Executive summary

SmartAg Ops Advisor is a web application for Indian agricultural operations. It helps users manage crop stock, assess local and government-reported mandi prices, estimate the net return from selling at different markets, arrange truck transport, coordinate bookings, and track deliveries. It also provides administrative controls, document handling, reports, notifications, and an agriculture-focused multilingual assistant.

The application has two separate market-price concepts:

1. **App-recorded prices** are manager-entered prices associated with the application's own market records. These are stored in rupees per kilogram and are used by the existing market comparison workflow.
2. **Government mandi prices** are imported from a dataset or the data.gov.in Agmarknet feed. These are stored and presented as published in rupees per quintal. They power the government price board and the market finder.

These feeds are distinct and should not be combined without an explicit unit and data-source conversion.

---

2. Product scope and roles

Roles

The application recognizes the following role identifiers:

Role	Main purpose
`farmer`	Maintain crop stock, compare markets, find mandi options, book transport, and track deliveries.
`truck_operator`	Maintain truck listings, respond to booking requests, manage trips, and share trip locations.
`market_manager`	Publish app-recorded crop prices for markets.
`buyer`	Use the market and produce workflows available to non-admin users.
`marketer`	Use the market and produce workflows available to non-admin users.
`admin`	Manage users, review operational metrics, and administer data such as mandi-feed synchronization.

Public signup is restricted to farmer and truck-operator roles. Other roles are assigned by an administrator. Backend authorization is authoritative; frontend route and navigation restrictions are supplementary user-interface controls.

Core journeys

Farmer selling workflow

1. Create or update crop stock, including quantity, pickup location, and optional production cost.
2. Review app-recorded market prices and compare estimated net proceeds, or use the government mandi board and market finder.
3. Select a truck and submit a booking for a destination market.
4. Follow booking status and trip location, then review recorded delivery and reporting information.

Truck-operator workflow

1. Complete the mandatory profile: contact, location, vehicle details, vehicle permit, and both sides of the driving licence.
2. Register a truck with its type, fares, status, and current location.
3. Review and accept or reject booking requests.
4. Progress accepted trips through pickup, transit, and delivery statuses.
5. Share live GPS location with the farmer every 15 seconds while the trip is picked up or in transit.

Administrator workflow

1. Review application overview metrics and user accounts.
2. Assign roles or remove accounts subject to safeguards.
3. Review users' uploaded verification documents.

4. Check mandi-data synchronization status and request a synchronization run.

---

3. Architecture

Components

Component	Implementation	Responsibility
Web client	React 18, Vite, React Router, Axios	Authenticated application interface, forms, maps, charts, and localization.
API	Python, FastAPI, Pydantic, SQLAlchemy	Authentication, access control, domain rules, integrations, and JSON APIs.
Relational database	SQLAlchemy-supported database; PostgreSQL is the documented deployment choice; SQLite connection support is present	User, operational, conversation, mandi, cache, and metrics persistence.
Map display	Leaflet and React Leaflet	Interactive market and trip maps.
AI assistant	Gemini API or a locally reachable Ollama service	Text responses; Gemini is also used for speech transcription.
Price feed	Local CSV dataset or data.gov.in Agmarknet resource	Government mandi-price import.
Geospatial services	OpenStreetMap Nominatim and public OSRM route service	Geocoding and road distance/time estimates for market finder.
Email	Mailtrap Email Sending API with optional SMTP fallback	OTP delivery for admin sign-in and password recovery.

Request and authentication flow

The browser client calls the API through the Axios service in `agri-project/frontend/src/services/api.js`. The backend base URL can be overridden by `VITE_API_URL`; the frontend defaults to the deployed Render API at `https://smartag-ops-backend.onrender.com`. After authentication, the client stores the bearer token and current user in browser local storage and sends the token in the `Authorization` header. The backend resolves the current user from the signed token and applies role checks on protected operations.

The frontend is a single-page application with route guards in `App.jsx`. The backend is the source of truth for identity and authorization. Do not treat hidden navigation items or frontend route guards as security boundaries.

Startup behavior

The FastAPI application is created in `backend/app/main.py`. At process import and startup it:

- Registers SQLAlchemy models and creates missing tables with `Base.metadata.create_all`.
- Runs lightweight compatibility changes for selected earlier schemas.
- Attempts configured default-admin initialization.
- Validates production security configuration.
- Installs CORS, response security headers, request metrics, and the background mandi synchronization scheduler.
- Registers the API routers and health endpoints.

The current schema setup is not a versioned migration framework. See known constraints before deploying schema-changing updates to an existing production database.

4. Repository map

```
.
├── README.md
├── SECURITY.md
├── LICENSE
├── docs/
│   ├── MASTER_DOCUMENT.md / .pdf
│   ├── PROMPT_DOCUMENT.md / .pdf
│   └── SmartAg_Work_Report.pdf
├── dataset/
│   └── tamil_nadu_tomato_mandi.csv
└── agri-project/
    ├── backend/
    │   ├── requirements.txt
    │   └── app/
    │       ├── core/       # settings, DB, email, security, geo, sync, metrics
    │       ├── models/     # SQLAlchemy entities
    │       ├── routers/    # HTTP endpoints and domain workflows
    │       └── schemas/    # Pydantic request/response models
    └── frontend/
        ├── package.json
        ├── vite.config.js
        └── src/
            ├── components/ # shared shell, maps, charts, dialogs
            ├── context/    # authentication and language state
            ├── hooks/      # browser interaction hooks
            ├── i18n/       # generated translations and message catalog
            ├── pages/      # route-level UI
            ├── services/   # API clients
            ├── styles/
            └── utils/
```

Main backend files:

- `backend/app/main.py`: application setup, middleware, startup and router registration.
- `backend/app/core/config.py`: typed settings loaded from the environment and `backend/.env`.
- `backend/app/core/database.py`: SQLAlchemy engine, session factory, and dependency.
- `backend/app/core/security.py` and `security_policy.py`: password/JWT helpers, access checks, request protections, headers, and production checks.
- `backend/app/core/mandi_sync.py`: dataset/API import and scheduled refresh.
- `backend/app/core/geo.py`: Nominatim geocoding and OSRM route caching.

Main frontend files:

- `frontend/src/App.jsx`: route definitions and role-gated layouts.
- `frontend/src/components/AppShell.jsx`: authenticated navigation and page shell.
- `frontend/src/context/AuthContext.jsx`: client-side auth state.
- `frontend/src/context/LiveShareContext.jsx`: the truck operator's 15-second GPS sender, which keeps running across pages and resumes after a reload; `components/LiveShareBar.jsx` shows its state on every page.
- `frontend/src/context/LanguageContext.jsx`: language selection and translation lookup.

- `frontend/src/services/`: API wrappers for authentication, operations, chat, and mandi/market-finder features.
- `frontend/src/pages/`: login, dashboard, crops, market prices, markets, bookings, tracking, reports, profile, chat, and administration screens.

---

5. User-facing features

Authentication and profile

- Email/password signup and sign-in.
- Admin sign-in requires an emailed one-time code after password verification.
- Forgot-password and OTP-based reset flow.
- Profile data includes user details and operator vehicle-profile fields. The Edit profile panel opens on demand; Save stores fields and staged document images together, and Reset discards changes.
- Administrator user management includes role updates and account removal (not your own account, not the last admin; a removed operator's trucks become inactive).
- Truck operators upload a vehicle permit and the driving licence front and back (JPEG, PNG, or WebP, up to 5 MB, checked by file content). Images are stored in the database and administrators can review them.

Mandatory fields for truck operators

Mandatory fields are marked with a red `*` and enforced by both the browser and the API. Requests that bypass the form are rejected with `422`.

Form	Mandatory fields
Profile	Full name, mobile number, village/town, vehicle type, vehicle number (3+ characters), vehicle capacity (> 0 kg), vehicle permit, licence front, licence back
Add truck	Vehicle number, truck type, status, base fare (≥ 0), rate per km (> 0), current location

Blank truck fields are no longer replaced with default fares or types. An operator can replace a saved permit or licence image but cannot delete it (`DELETE /api/auth/me/documents/{type}` returns `400`). Farmers and other roles are not affected by these rules.

Dashboards

- **Farmer:** key figures, crop stock, a map of nearby markets with the nearest three, a revenue chart of sales recorded after delivery over the last six months (empty state when there are none), and available trucks.
- **Truck operator:** trip and request overview.
- **Admin:** user counts by role, an AI assistant usage chart, and an API request chart, each for 24 hours, 7 days, or 30 days. Farmer workflow pages (crops, market finder, trucks, market prices, tracking, bookings) are hidden from and blocked for administrators.

Crop stock and app-recorded market prices

Farmers can create, update, list, and delete their own crop-stock records. Records can include a crop name, available quantity, pickup location, and production cost per kilogram.

The existing market board stores manager-entered prices per kilogram. It provides latest-price and previous-price comparisons, trend percentages, stale price flags, and a market comparison endpoint. Its net-proceeds

estimate applies the configured selling-loss, transport, loading, labour, commission, and packaging assumptions. If a crop's production cost is known, the estimate also includes estimated profit.

The legacy comparison workflow uses the market record's configured distance and a standard transport rate. Its assumptions are returned by the API; these figures are estimates, not a transport quote or guaranteed sale outcome.

Government mandi board

The government price board is backed by `MandiPrice`, not the app-recorded `MarketPrice` table. It supports:

- Latest-date prices and filters for state, district, market, and commodity.
- Text search and selectable sort order.
- Pagination and summary counts/prices.
- History for a selected state, market, and commodity.
- A status view of the selected source and the latest synchronization.
- Admin-triggered background synchronization.

Published prices are kept in rupees per quintal. The included sample dataset is a Tamil Nadu tomato CSV snapshot. It is an input dataset, not a continuously updated feed.

Market finder

The market finder combines the latest government modal prices for a commodity with the user's origin and estimated delivery costs. The user can provide coordinates or a place name; the service can also try a crop pickup location or the profile location. Coordinates are restricted to India.

Market coordinates are resolved in the background and cached. The finder shortlists markets using straight-line distance and published price, then requests/caches road distance and duration for the shortlist. A suitable available truck in the app can supply the rate; otherwise the configured standard transport base and per-kilometre values are used. The result reports the cost breakdown, estimated net, and optional production-cost-based profit. Market location may fall back to an approximate district center when a precise market location cannot be established.

This is a planning estimate based on the feed's available prices and public geospatial services. It is not a booking, quote, guarantee of price, or recommendation that replaces the user's judgment.

Trucks, bookings, delivery tracking, and reports

Truck operators maintain fleet entries and availability; farmers can find trucks and request capacity. Booking records connect the farmer, truck, destination market, optional crop, quantity, status, and delivery information. The API enforces participant and role checks on list and update operations. Trip status and location updates are used by tracking pages and maps.

Live trip location

Location is shared only with the operator's consent and only while a trip is `picked_up` or `in_transit`.

1. The operator ticks the consent box on Live Tracking. The app then sends one GPS fix every **15 seconds** through `PATCH /api/bookings/{id}/location`, even when the position has not changed.
2. Sharing continues while the operator uses other pages, resumes after a page reload, and keeps the screen awake where the browser allows it. A bar on every page shows the time since the last update and a **Stop sharing** button.
3. Each fix is stored in `booking_location_points`; the newest 960 per trip (about four hours) are kept.

4. The farmer's tracking page reads `GET /api/bookings/{id}/track` every five seconds and shows the truck, the route travelled, and "Live · updated N s ago". After 45 seconds without an update it warns that the truck may have lost signal.
5. Sharing stops when the operator presses Stop, the trip is delivered, or the server refuses an update. Stopping or delivery deletes the route and hides the truck from the farmer.

A website cannot send location after the app or tab is closed, or while the phone suspends the browser; that would need a native app.

Dashboard and report endpoints summarize operational records. Notifications can be listed and marked read individually or in bulk. Search provides a cross-feature lookup through the backend search endpoint.

Chat and voice input

The AI assistant is intended for agriculture-related questions, persists user conversations and messages, and can stream responses. Conversation ownership is checked by the API. The assistant's system prompt constrains its subject matter, asks it not to invent prices/scheme/weather information, and requests concise answers in the selected language.

If the main Gemini model is busy or has not started answering within `GEMINI_FIRST_TOKEN_SECONDS` (20 s), the backend retries with `GEMINI_FALLBACK_MODEL`. The browser shows "Still thinking…" after 12 seconds and stops waiting after 75 seconds without data.

The client can select Gemini or Ollama for text chat. Gemini requires `GEMINI_API_KEY`; Ollama requires the configured local service and model. Voice input is uploaded for Gemini transcription and is not the same as sending a voice response. Provider status is exposed through the chat API.

Languages and route-level screens

The generated UI language catalog contains English, Hindi, Tamil, Telugu, Kannada, Malayalam, Marathi, Bengali, Gujarati, and Punjabi. The market-price board and market-finder controls, results, and map controls are included in those translations. Market, district, state, commodity, and variety names come from the source data and are not translated. External provider error details may also remain in English.

Frontend routes include:

Route	Screen	Access
`/login`, `/forgot-password`	Sign-in and password recovery	Public
`/dashboard`, `/reports`, `/profile`, `/chat`	Main user workspace	Authenticated users
`/crops`, `/markets`, `/market-prices`, `/trucks`, `/tracking`, `/bookings`	Farmer/market operations	Non-admin application roles
`/booking-requests`, `/trips`, `/notifications`	Operator trip management	Truck operators; admin route guard also permits these routes
`/admin`	Administration	Admin

The root route redirects to the dashboard for signed-in users and to login otherwise. Unknown paths redirect through the root route.

---

6. Data model

The SQLAlchemy model modules are registered at application startup. Principal tables include:

Entity/table	Purpose
`users`	Credentials, role, profile, vehicle profile, and authentication lock/session timestamps.
`markets`	Application's operational markets and distance/ETA metadata.
`crop_stocks`	User-owned crops, quantity, pickup details, and production-cost data.
`market_prices`	Manager-entered per-kilogram market-price observations.
`trucks`	Operator-owned vehicle, capacity, fare, and availability data.
`bookings`	Farmer-to-truck transport requests, status, delivery, and latest location.
`booking_location_points`	Live GPS trail per booking (latitude, longitude, accuracy, time); capped at 960 points and cleared on stop or delivery.
`notifications`	Per-user operational notifications and read state.
`user_documents`	Uploaded user verification documents and review metadata.
`chat_conversations`, `chat_messages`	User-owned assistant conversations and messages.
`password_reset_otps`, `admin_login_otps`	OTP state for password recovery and admin login.
`mandi_prices`, `mandi_sync_runs`	Imported government prices and sync execution history.
`geocode_cache`, `route_cache`	Cached geocodes and road-route results for market finder.
`api_request_stats`	Aggregated API request metrics.

Operational entities use foreign keys for ownership and relationships. User records own crops, documents, and conversations; bookings reference farmers, trucks, and markets. Mandi data is stored separately from manager-entered market prices.

---

7. API reference

All API paths are relative to the backend origin. Unless described as a public authentication operation, endpoints require a bearer token. Role and ownership requirements are enforced by the backend. Request/response schemas and development OpenAPI docs are the definitive source for payload details.

Prefix/path	Methods and purpose
`/`, `/api/health`	`GET` service welcome and health status.
`/api/auth`	Signup, login, admin login OTP verification/resend, forgot password, OTP verification, password reset, current-user read/update, admin user list/role update/delete. Also contains legacy role-check area endpoints.
`/api/auth/me/documents`	`GET` list, `GET` by document type, `PUT` upload/replace, `DELETE` own documents.
`/api/auth/users/{user_id}/documents`	Admin document-list and per-type review access.
`/api/admin`	`GET /overview` administrator overview.
`/api/crops`	`GET`, `POST`, `PATCH /{crop_id}`, `DELETE /{crop_id}` for crop stock.

Prefix/path	Methods and purpose
`/api/markets`	Market listing, individual market and market-price views, price listing/creation, price crop options, and `GET /compare`.
`/api/mandi-prices`	`GET` price list; `GET /filters`, `/history`, `/status`; admin-only `POST /sync`.
`/api/market-finder`	`GET /commodities`, `GET` market ranking, and `GET /route` for route estimates.
`/api/trucks`	`GET` list/detail, `POST` create, `PATCH /{truck_id}` update, `DELETE /{truck_id}` delete.
`/api/bookings`	`GET` list/detail, `POST` create, and `PATCH` status, delivery, and location updates. `GET /{id}/track` returns the live GPS trail (`?since=` for newer points only) and the 15-second interval.
`/api` dashboard	`GET /dashboard`, `/reports/summary`, `/notifications`, `/search`; `PATCH` notification read and read-all actions.
`/api/chat`	Provider status, voice transcription, conversation list/create/read/rename/delete, message submit, and streamed message submit.

The server provides `/docs`, `/redoc`, and `/openapi.json` outside production. These API documentation endpoints are disabled when `ENVIRONMENT=production`.

---

8. Configuration reference

The backend loads settings from environment variables and `backend/.env`. `backend/.env.example` is a template only. Keep the actual `.env` out of version control. The following names match the current settings model; empty defaults mean a value must be provided when the associated feature is used.

Runtime, database, and security

Variable	Purpose/current default
`APP_NAME`	API title; default `SmartAg Ops Advisor API`.
`ENVIRONMENT`	`development`; set to `production` for stricter startup and transport behavior.
`SECRET_KEY`	Signing/HMAC secret; replace the development placeholder with a strong random secret.
`ALGORITHM`	JWT algorithm, currently `HS256`.
`ACCESS_TOKEN_EXPIRE_MINUTES`	Token lifetime; default 1440 minutes.
`DATABASE_URL`	SQLAlchemy connection URL; set to the deployed database.
`FRONTEND_URL`	Allowed frontend origin; default `http://localhost:5173`.
`ADDITIONAL_FRONTEND_ORIGINS`	Comma-separated explicit extra CORS origins; currently defaults to `https://agri-project-frontend-2bf3.onrender.com`.
`PASSWORD_MIN_LENGTH`	Minimum password length; default 8.
`LOGIN_MAX_FAILURES`	Failed login threshold; default 5.
`LOGIN_LOCKOUT_MINUTES`	Lock duration after threshold; default 15 minutes.

Variable	Purpose/current default
`MAX_REQUEST_BYTES`	Maximum request body; default 12 MiB.
`SEED_DEMO_DATA`	Declared with default false, but no seeding behavior currently references it.

Email and OTP

Variable	Purpose/current default
`ADMIN_EMAIL`, `ADMIN_DEFAULT_PASSWORD`	Optional first-admin provisioning values; defaults are empty. Configure securely if using startup provisioning.
`ADMIN_NAME`	Initial admin display name; default `Admin`.
`OTP_EXPIRE_MINUTES`	OTP lifetime; default 10 minutes.
`OTP_RESEND_SECONDS`	Resend cooldown; default 60 seconds.
`OTP_MAX_ATTEMPTS`	Verification attempts; default 5.
`RESET_TOKEN_EXPIRE_MINUTES`	Password-reset token lifetime; default 15 minutes.
`MAILTRAP_API_TOKEN`	Optional Mailtrap token; empty by default.
`MAILTRAP_API_URL`	Mailtrap API URL.
`MAILTRAP_FROM_EMAIL`, `MAILTRAP_FROM_NAME`	Mail sender identity.
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_FROM`	Optional SMTP fallback.
`SMTP_USE_TLS`, `SMTP_USE_SSL`	SMTP transport settings; defaults true and false respectively.

AI

Variable	Purpose/current default
`GEMINI_API_KEY`	Gemini API credential; empty by default.
`GEMINI_MODEL`	Text model; default `gemini-flash-latest`.
`GEMINI_FALLBACK_MODEL`	Chat model used when the main model is busy or slow; default `gemini-flash-lite-latest`.
`GEMINI_FIRST_TOKEN_SECONDS`	Wait for the main model's first output before falling back; default 20 seconds.
`GEMINI_SPEECH_MODEL`	Voice transcription model; default `gemini-flash-lite-latest`.
`GEMINI_TIMEOUT_SECONDS`	Gemini request timeout; default 180 seconds.
`AI_MAX_OUTPUT_TOKENS`	Text response limit; default 200.
`AI_MAX_HISTORY_MESSAGES`	Prior messages included in provider context; default 6.
`AI_MAX_INPUT_CHARS`	Per-message text clipping limit; default 1200 characters.
`OLLAMA_BASE_URL`	Local Ollama URL; default `http://127.0.0.1:11434`.
`OLLAMA_MODEL`	Ollama model name; default `llama3.2:3b`.
`OLLAMA_TIMEOUT_SECONDS`	Ollama timeout; default 120 seconds.

Price calculation and mandi data

Variable	Purpose/current default
`TRANSPORT_RATE_PER_KM`	Standard transport rate; default 28.0.
`LOADING_COST`	Standard base/loading cost; default 500.0.
`MARKET_LOSS_PCT`	Assumed produce loss; default 2.0%.
`LABOUR_COST_PER_KG`	Labour assumption; default 0.4.
`COMMISSION_PCT`	Commission assumption; default 2.0%.
`PACKAGING_COST_PER_KG`	Packaging/handling assumption; default 0.25.
`STALE_PRICE_HOURS`	Age after which manager-entered prices are flagged stale; default 48 hours.
`MANDI_SOURCE`	`dataset` (default) or `api`.
`MANDI_DATASET_DIR`	Optional CSV folder; default is the repository-level `dataset/` directory.
`DATA_GOV_API_KEY`	API key required when `MANDI_SOURCE=api`.
`DATA_GOV_MANDI_RESOURCE`	data.gov.in resource identifier; default targets the Agmarknet price feed.
`MANDI_SYNC_INTERVAL_HOURS`	Minimum interval between successful API syncs; default 6 hours.
`MANDI_RETENTION_DAYS`	Rolling retention window counted back from the newest published date; default 30 days.

Frontend

Variable	Purpose
`VITE_API_URL`	Backend origin used by the frontend; defaults to `https://smartag-ops-backend.onrender.com`.

Vite variables are embedded in the client build and must never contain server credentials.

---

9. Local development

Requirements

- Python 3.12 or a compatible Python version supported by the installed dependencies.
- Node.js 18 or newer and npm.
- PostgreSQL for the documented database setup, or a compatible local database URL for development.
- Optional: Gemini key for Gemini assistant/transcription; Ollama installed and running for local model use.

Backend

From the repository root, in PowerShell:

```
Set-Location agri-project\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
```

Edit `backend/.env` with a database URL and a newly generated local `SECRET_KEY`. Set email, AI, or mandi-feed credentials only for features you intend to exercise. Create the database before starting the server.

Run the API:

```
uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

The API is available at `http://127.0.0.1:8000`; development docs are at `http://127.0.0.1:8000/docs`.

Frontend

In another PowerShell terminal:

```
Set-Location agri-project\frontend
npm install
Copy-Item .env.example .env
npm run dev
```

Open `http://localhost:5173`. The frontend `.env.example` and built-in development default use `https://smartag-ops-backend.onrender.com`. Set `VITE_API_URL` only when intentionally testing against a different API.

First administrator

Public signup cannot create an administrator. To provision the initial admin, set `ADMIN_EMAIL`, `ADMIN_DEFAULT_PASSWORD`, and optionally `ADMIN_NAME` in the backend environment before startup. The initializer creates the account if it does not exist and does not reset the password of an existing account. Admin authentication then requires the email OTP flow. Use a strong unique password and configure a working email provider before relying on admin access.

---

10. Data and external services

Mandi source modes

`MANDI_SOURCE=dataset` imports CSV files from `MANDI_DATASET_DIR` or the repository's `dataset/` folder. CSV headers are case-insensitive and must include:

```
state,district,market,commodity,variety,grade,arrival_date,min_price,max_price,modal_price
```

Arrival dates use `DD/MM/YYYY`. Prices remain in rupees per quintal. The scheduler checks for changed dataset files periodically and imports them; changed file signatures are remembered by the running process.

`MANDI_SOURCE=api` downloads pages from data.gov.in using `DATA_GOV_API_KEY` and `DATA_GOV_MANDI_RESOURCE`. Configure the key through the server environment; it is redacted from synchronization logs. The scheduler checks whether the configured refresh interval is due. A manual sync is available to admins through `POST /api/mandi-prices/sync`.

Both modes record synchronization runs, upsert duplicate observations by market/commodity/date identity, and prune records older than the configured retention window relative to the newest loaded arrival date. A failed import is recorded as a failed run; review the status endpoint and server logs.

Geospatial services

- Nominatim is queried for place and market coordinates, with a one-request-per- second throttle and a database cache.

- OSRM is used for driving distance and duration; route results are cached.
- The client displays Leaflet maps and third-party map tiles.
- These are external public services. Availability, accuracy, acceptable use, rate limits, and terms are outside the application's control.

AI and email dependencies

Gemini requires a server-side key. Ollama must be reachable at `OLLAMA_BASE_URL` with the configured model available. OTP email first uses the Mailtrap API when configured, with optional SMTP fallback; if neither is configured, local-development behavior may log an OTP to backend output. Do not use console-delivered OTPs in production.

---

11. Security and operations

The detailed security requirements and controls are maintained in SECURITY.md. Key operational points:

- Use HTTPS in production; set `ENVIRONMENT=production`, a strong random `SECRET_KEY`, and an HTTPS `FRONTEND_URL`.
- Keep `.env` files, API keys, database credentials, OTPs, and user documents out of source control and public logs.
- The backend enforces role and record ownership checks; do not weaken them to accommodate frontend-only behavior.
- CORS allows only `FRONTEND_URL`, the explicit `ADDITIONAL_FRONTEND_ORIGINS` list, and local development origins outside production. Add each deployed frontend origin exactly; do not use wildcard origins.
- Production configuration disables FastAPI interactive API documentation and checks for weak secrets.
- Security headers are added by the API and Vite development/preview configuration. Production frontend hosting must also send the required response headers; see `SECURITY.md`.
- Rate limiting is maintained in process memory. A multi-worker or multi-instance deployment needs a shared limiter to enforce global limits.
- Uploaded user documents are protected resources. Restrict administrative access and use secure database backups.
- Request metrics and application logs may contain operational metadata; set appropriate access and retention controls.

Production deployment checklist

1. Provision PostgreSQL and create a dedicated least-privilege application account. Configure a TLS database connection as required by the provider.
2. Set all backend environment values through the deployment platform's secret store; use a strong secret key and production frontend URL.
3. Configure the frontend build with the public API origin only. Do not put private service credentials in `VITE_*` values.
4. Configure and test OTP email before enabling administrator or password recovery workflows.
5. Select the mandi source and verify `GET /api/mandi-prices/status` after startup.
6. Verify Nominatim/OSRM network access and configure a supported production mapping/routing service if public endpoints do not meet expected service requirements.
7. Serve the frontend and API over HTTPS, apply security headers at the hosting layer, and restrict database and API network access.

8. Back up the database and rehearse schema-upgrade and restore procedures.
9. Monitor health, API metrics, synchronization status, database capacity, and external integration failures.

---

12. Build and validation

Frontend production build:

```
Set-Location agri-project\frontend
npm run build
```

Backend syntax compilation:

```
python -m compileall -q agri-project\backend\app
```

The frontend package currently defines `dev`, `build`, and `preview` scripts; it does not declare a test or lint script. Add or run focused test suites when available. A successful production build verifies bundling, not runtime integration with the database, email provider, AI provider, or map services.

---

13. Known constraints and maintenance notes

- **Schema evolution:** startup uses `create_all` and hand-written compatibility changes for selected historical fields. There is no versioned migration toolchain in the current dependency manifest. Plan explicit, reversible migrations and backups for production changes; do not assume `create_all` alters existing tables.
- **Database portability:** SQLite connection arguments are handled in the database module, and mandi upserts select PostgreSQL/SQLite dialects. Nevertheless, production setup in the README targets PostgreSQL and one legacy migration path contains SQLite-specific SQL. Validate the complete startup path against the actual production database engine.
- **Mandi freshness:** dataset mode only changes when CSV files change. API mode depends on data.gov.in availability and key quotas. A successful sync does not mean every market has a current arrival record.
- **Price units and semantics:** app-recorded prices are per kilogram; government prices are per quintal. Different source dates and market names are possible; display source, date, and units when comparing or exporting.
- **Market finder estimates:** geocoding can be approximate, road routing may be unavailable, a suitable app truck may not exist, and prices can change. Explain assumptions and uncertainty in user-facing contexts.
- **External dependencies:** Gemini, Ollama, Mailtrap/SMTP, data.gov.in, Nominatim, OSRM, and map tiles can fail independently. The API and UI should surface these as unavailable or stale rather than presenting old results as current.
- **Rate-limit scope:** the in-memory limiter is per process and is not a distributed abuse-control solution.
- **Live location:** updates depend on the operator's browser staying open with location permission; phones may pause background tabs. The stored trail grows by four rows per minute per sharing trip and is capped per booking.
- **Deployment target:** the frontend `.env` points at the Render backend, so backend changes reach the running app only after a push and redeploy.
- **Localization:** ten languages are represented in the translation catalog, but recently added or hard-coded UI text can still fall back to English.

- **Generated translation files:** `frontend/src/i18n/translations.js` is generated by `scripts/generate-translations.mjs`; make source-copy changes in the appropriate message source and regenerate rather than editing generated output by hand.

Further reading

- Prompt Document (`docs/PROMPT_DOCUMENT.pdf`) — every request that shaped the app, with reusable prompts.
- Project README — original quick-start overview.
- Security policy — detailed security rules and controls.
- Backend environment template — backend setting names and safe example values.
- Frontend environment template — public API URL configuration.